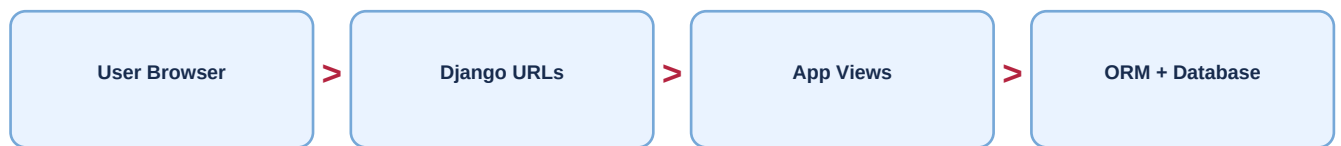


VIVA PREPARATION HANDBOOK

ZENZEE

Django-based Youth Streetwear E-commerce Platform

One-line introduction: ZENZEE is a modular Django fashion e-commerce platform with authentication, product discovery, cart, wishlist, checkout, test payments, orders, reviews, rule-based styling recommendations and an admin dashboard.



Django 5.2.16 | 8 custom apps | Razorpay test mode | Render deployment

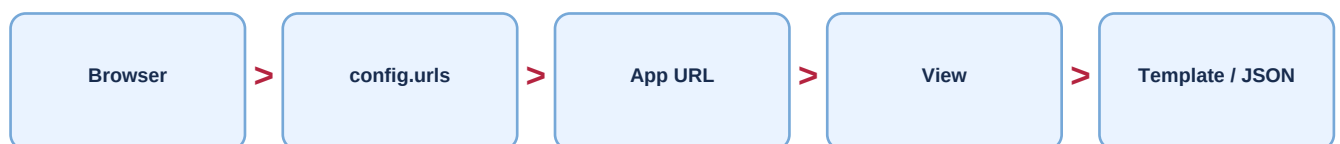
1. Project Snapshot

Backend	Python and Django MTV architecture
Frontend	Django templates, HTML, CSS, Bootstrap and JavaScript
Database	Django ORM with SQLite locally; DATABASE_URL support for PostgreSQL
Apps	accounts, products, cart, wishlist, orders, reviews, ai and dashboard
Deployment	GitHub to Render, Gunicorn, WhiteNoise, migrations and collectstatic

2. Modules and Responsibilities

accounts	Custom user, registration, login, logout and profile
products	Categories, brands, sizes, products, images, pricing and stock relation
cart	Guest session carts, user carts, quantities, subtotal and shipping
wishlist	Authenticated users save or remove products; uniqueness enforced
orders	Checkout, Order, OrderItem, COD, Razorpay and order history
reviews	Logged-in users submit product ratings and comments
ai	Rule-based recommendations and chat history API
dashboard	Staff-only users, products, orders, revenue and recent orders

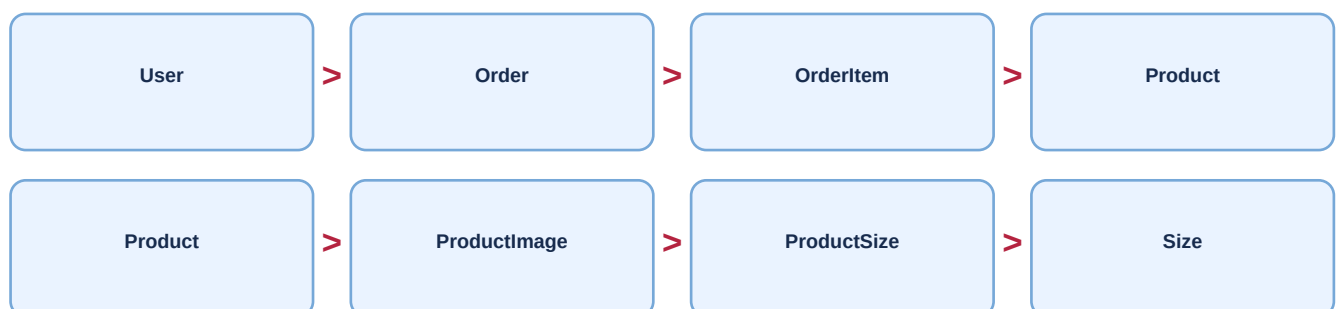
3. Architecture and Request Flow

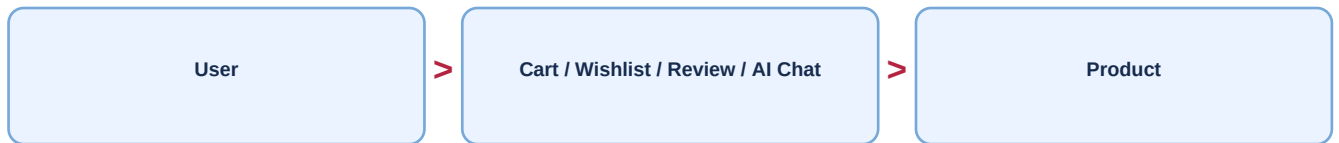


Example: / -> accounts.views.home_view -> Product ORM query -> home.html

Database path: View -> Model -> Django ORM -> SQLite or PostgreSQL -> response.

4. Database Relations



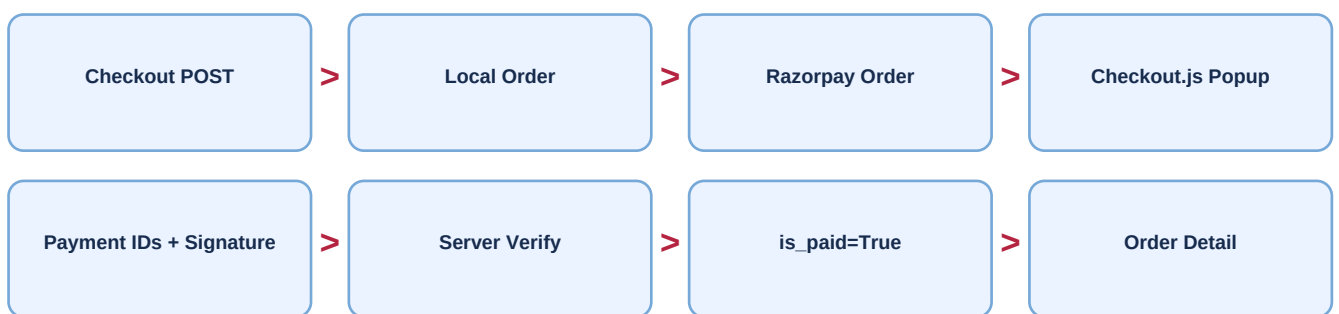


Design answer: OrderItem stores the purchase-time price, so later product price changes do not change old orders.

5. Main Inner Workings

Product pricing: `get_effective_price()` uses `discount_price` only when it exists and is lower than price. **Shipping:** INR 99 when subtotal is 1,499 or below, free above 1,499. **Cart:** authenticated users use a user relation; guests use a session key. **Checkout:** validate form, create Order, create OrderItems, clear cart, then COD confirmation or Razorpay page.

6. Razorpay Payment Flow



Q: Why multiply the amount by 100?

Razorpay expects paise, the smallest currency unit. INR 100 becomes 10,000 paise.

Q: Why not trust browser payment status?

Browser values can be modified. The server verifies the signature with the Razorpay secret key.

Q: Is this live payment?

No. It is Razorpay sandbox/test mode, so the viva demonstration should not charge real money.

7. Likely Viva Questions and Answers

Q: Why Django?
Django provides routing, ORM, authentication, forms, templates, admin, middleware and security in one mature framework.
Q: What is MTV?
Model stores data, Template presents the UI and View processes requests and responses. It is Django's version of MVC.
Q: What is manage.py?
The command-line entry point for runserver, migrate, makemigrations, test and collectstatic.
Q: What is ORM?
Object Relational Mapping maps Python model classes to database tables and provides Python-based queries.
Q: Why a custom user model?
ZENZEE needs phone number, profile picture and bio in addition to Django authentication fields.
Q: What are migrations?
Version-controlled schema changes. makemigrations creates files and migrate applies them.
Q: What is CSRF?
Cross-Site Request Forgery protection blocks unwanted state-changing requests made using a user session.
Q: What does login_required do?
It protects profile, order history, wishlist and review actions from anonymous users.
Q: What does WhiteNoise do?
It serves collected static CSS, JavaScript and assets directly from the Django deployment.
Q: Why cd backend on Render?
manage.py and config are inside backend, so Gunicorn must start there to import config.wsgi.
Q: Why did Render return 500?
The fresh deployment database had no schema because migrations were not run. Deployment now runs migrations before Gunicorn.
Q: What is the AI feature?
A rule-based recommender that filters products using words such as hoodie, oversized, streetwear, cargo and budget.

8. SDLC Answer



Requirements	Fashion store with accounts, browsing, cart, wishlist, checkout, payment, orders, reviews and admin statistics.
Design	Separated apps, relational models, routes, responsive templates and payment sequence.
Development	Python/Django, ORM, forms, templates, Bootstrap, JavaScript and Razorpay SDK.
Testing	Django TestCase, route smoke tests, migration and production configuration checks.
Maintenance	Improve database durability, storage, security, test coverage and monitoring.

9. Testing Status

Verified: 12 automated Django tests pass when run from backend. Live smoke checks for homepage, products, login, register, AI Stylist and cart returned HTTP 200. Django system check passes and migrations are applied.

User/superuser creation, account pages, profile redirect, AI API, checkout rendering and product discount calculations.	Cart add/remove, wishlist, reviews, complete order creation, Razorpay valid/invalid signatures, dashboard permissions and browser checkout.

Correct command: cd backend then python manage.py test. Root execution can incorrectly show zero discovered tests because the project is nested.

10. Problems, Solutions and Limitations

Template syntax error	Corrected invalid Django conditional syntax in checkout template.
Render import error	Started Gunicorn from backend.
Render 500	Added deployment-time migrations before starting the app.
Static warning	Added collectstatic and WhiteNoise support.
SQLite durability	Use managed PostgreSQL for real production persistence.
Media durability	Use cloud object storage for uploaded images.
Payment security	Keep signature verification; disable demo fallback and validate gateway order ownership in production.
Order privacy	Restrict order detail to the order owner or staff.

11. Demo Order

1. Homepage and one-line introduction.
2. Products, search/filter and product detail.
3. Add product and size to cart; explain price and shipping.
4. Login, wishlist and profile protection.
5. AI Stylist query: "oversized hoodie under 1500".
6. Checkout: explain COD and Razorpay test mode.
7. COD order and order history.
8. Explain Razorpay signature verification without claiming a real charge.
9. Staff dashboard, deployment and limitations.

12. Final 30-Second Answer

Q: How would you summarize the project?

ZENZEE is a modular Django e-commerce application for youth streetwear. Eight apps separate business responsibilities. Django ORM manages users, products, cart, orders, reviews and wishlist data. Checkout supports COD and Razorpay sandbox payments, while the server verifies the payment signature. The AI stylist is rule-based. The application is deployed on Render using Gunicorn and WhiteNoise. Twelve automated tests and live route smoke checks passed; future work includes PostgreSQL, cloud media, stronger order authorization and broader integration testing.